CS509 -PG Software Lab

Aryan Shrivastva (2026CSM1007) Akash Maurya (2026CSM1024)

Assignment 3(Buddy)

Assignment Overview

This assignment is completed as a Buddy/Pair Assignment by two students in a shared repository.
The assignment implements two algorithms:

Gradient Descent

- An iterative optimization algorithm used to find a minimum of a mathematical function.
- Uses the derivative of the function to determine the direction of movement.
- Updates the current value of x using a learning rate.
- Stops when the magnitude of the derivative becomes smaller than the given tolerance or when the maximum number of iterations is reached.

Maxflow-Mincut

- Computes the maximum amount of flow that can be sent from a source vertex to a sink vertex.
- Uses directed edges with positive integer capacities.
- The implementation uses Dinic's Maximum Flow algorithm.
- After computing maximum flow, the minimum cut is obtained from the final residual graph.
- The source side contains vertices reachable from the source in the residual graph.
- The sink side contains the remaining vertices.
- The maximum-flow value must be equal to the minimum-cut capacity.

Language and Environment

- The complete repository has been implemented using modern C++ following the laboratory guidelines.
- **Programming Language**: C++
- **Compiler:** GNU g++ Compiler
- **Build Tool:** Makefile
- **Operating System:** Windows 11

Repository Structure

```
assignment_03/
│
├── driver/
│   ├── driver_gradient_descent.cpp
│   └── driver_maxflow_mincut.cpp
│
├── src/
│   ├── gradient_descent.hpp
│   ├── gradient_descent.cpp
│   ├── Maxflow_Mincut.hpp
│   └── Maxflow_Mincut.cpp
│
├── tests/
│   ├── gradient_descent/
│   │   ├── gd_01.txt
│   │   ├── gd_02.txt
│   │   ├── gd_03.txt
│   │   └── ...
│   │
│   └── maxflow_mincut/
│       ├── mf_01.txt
│       ├── mf_02.txt
│       ├── mf_03.txt
│       └── ...
│
├── outputs/
│   ├── gradient_descent/
│   │   ├── gd_01_output.txt
│   │   ├── gd_02_output.txt
│   │   └── ...
│   │
│   └── maxflow_mincut/
│       ├── mf_01.txt
│       ├── mf_02.txt
│       └── ...
│
├── executables/
│   ├── gradient_descent.exe
│   └── maxflow_mincut.exe
│
└── README.md
```

Common CSR Component

```
common/
└── csr/
    │
    ├── README.md
    │
    ├── src/
    │   ├── CSR.hpp
    │   ├── CSR.cpp
    │   ├── driver_csr.hpp
    │   └── driver_csr.cpp
    │
    ├── test_CSR/
    │   ├── csr_test_01.txt
    │   ├── csr_test_02.txt
    │   ├── csr_test_03.txt
    │   └── ...
    │
    └── outputs/
        └── graph/
```

The Maxflow-Mincut implementation uses the common CSR component to store the input directed capacity graph.

Assignment Objectives

- Implement Gradient Descent for numerical optimization.
- Calculate the value of a polynomial function.
- Calculate the derivative of the polynomial function.
- Perform iterative Gradient Descent updates.
- Detect convergence using the specified tolerance.
- Implement the Maxflow-Mincut problem.
- Use Dinic's algorithm for maximum flow.
- Use a residual graph internally during the flow computation.
- Obtain the minimum cut from the final residual graph.
- Ensure that maximum-flow value equals minimum-cut capacity.
- Use CSR representation for the Maxflow-Mincut input graph.
- Measure only algorithm execution time.
- Test the algorithms on the prescribed test cases.
- Generate separate output files for every test case.

Algorithm Comparison

Property	Gradient Descent	Maxflow-Mincut
Problem	Numerical optimization	Maximum flow and minimum cut
Input	Polynomial coefficients and parameters	Directed capacity graph
Main Technique	Iterative gradient update	Dinic's algorithm
Graph Required	No	Yes
CSR Used	No	Yes
Main Operation	Derivative-based update	BFS + DFS on residual graph
Output	Final x and f(x)	Maximum flow and minimum cut
Stopping Condition	Tolerance / max iterations	No augmenting path remains
Main Working Space	O(1) apart from input	O(V + E) approximately
Execution	Iterative numerical method	Network-flow algorithm

Gradient Descent

Gradient Descent is an iterative optimization method used to find a minimum of a function. For a function f(x), the derivative f'(x) determines the direction in which the function changes. The implementation updates x using:

x = x - learningRate × derivative

A positive derivative causes x to move in the negative direction, while a negative derivative causes x to move in the positive direction.

- ## Gradient Descent Algorithm

  For every test case:

  a. Read the polynomial degree.
  b. Read the polynomial coefficients.
  c. Read the initial value of x.
  d. Read the learning rate.
  e. Read the convergence tolerance.
  f. Read the maximum number of iterations.
  g. Calculate the derivative at the current x.
  h. Check whether the absolute derivative is smaller than or equal to the tolerance.
  i. If the condition is satisfied, mark the algorithm as converged.
  j. Otherwise, update x using the Gradient Descent formula.
  k. Increment the iteration count.
  l. Repeat until convergence or the maximum number of iterations is reached.
  m. Calculate the final function value.
  n. Report the final x, final f(x), number of iterations, convergence status, and execution time.

Maxflow-Mincut

The Maxflow-Mincut problem determines the maximum amount of flow that can be sent from a source vertex s to a sink vertex t without exceeding the capacity of any directed edge. The corresponding minimum cut divides the vertices into a source side and a sink side, such that the source belongs to the source side and the sink belongs to the sink side. The implementation uses Dinic's Maximum Flow algorithm.

- ## Dinic Algorithm Steps
    - Convert the CSR graph into a residual graph.
    - Add every original directed edge with its capacity.
    - Add a reverse residual edge with initial capacity zero.
    - Run BFS from the source.
    - Construct the level graph.
    - If the sink is unreachable, maximum flow has been obtained.
    - Initialize the current-edge array.
    - Use DFS to send flow through the level graph.
    - Update forward and reverse residual capacities.
    - Repeat DFS until no more flow can be sent.
    - Construct another level graph using BFS.
    - Repeat until the sink becomes unreachable.
    - The accumulated flow is the maximum-flow value.

Minimum Cut

After the maximum flow has been calculated, the final residual graph is used to determine the minimum cut. A BFS/graph traversal starts from the source and follows only edges with capacity > 0. All reachable vertices form the source side; all unreachable vertices form the sink side.

Minimum Cut Edges

An original edge is a cut edge when u is in the source side and v is in the sink side. The capacity of every such original edge is added to obtain the minimum-cut capacity. The implementation therefore reports:

- Source side
- Sink side
- Cut edges
- Minimum cut capacity

Gradient Descent Result Table

The following table summarizes the measured results obtained from the test cases; every listed case converged successfully.

Test File	Degree	Initial x	Learning Rate	Iterations	Final x	Final f(x)	Converged	Time
gd_01.txt	2	0	0.10	5,000	3	0	true	0.014100 ms
gd_02.txt	4	2	0.02	10,000	0	0	true	0.047400 ms
gd_03.txt	6	2	0.02	20,000	0	0	true	0.040300 ms
gd_04.txt	8	2	0.01	50,000	0	0	true	0.184300 ms
gd_05.txt	10	2	0.005	100,000	0	0	true	0.454200 ms

Maxflow-Mincut Result Table

The following table summarizes the measured Maxflow-Mincut results; status is Pass when Maximum Flow = Minimum Cut Capacity.

Test File	V	E	Source	Sink	Max Flow	Min Cut	Time	Status
mf_06.txt	6	10	0	5	23	23	0.0086 ms	Pass
mf_10.txt	10	15	0	9	8	8	0.0348 ms	Pass
mf_100.txt	100	200	0	99	83	83	0.0971 ms	Pass
mf_1000.txt	1000	1500	0	999	36	36	0.8122 ms	Pass
mf_10000.txt	10000	12000	0	9999	17	17	6.1367 ms	Pass
mf_50000.txt	50000	55000	0	49999	3	3	13.6148 ms	Pass
mf_100000.txt	100000	120000	0	999999	40	40	100.234 ms	Pass

Observations

- **Gradient Descent**
  - Gradient Descent successfully converged for all the test cases listed in the result table.
  - The number of iterations increases across the test cases, from 5,000 iterations in gd_01.txt to 100,000 iterations in gd_05.txt.
  - The execution time also generally increases as the number of iterations increases, from 0.014100 ms for gd_01.txt to 0.454200 ms for gd_05.txt.
  - The polynomial degree increases from 2 to 10 across the test cases, which also increases the amount of computation required for calculating the function derivative.
  - The learning rate decreases from 0.10 to 0.005 as the test cases become larger.
  - Despite the different polynomial degrees, learning rates, and iteration limits, all the listed test cases reached a converged solution.
  - Overall, the results show that the execution time increases with the computational workload while the implementation successfully converges for the tested configurations.

- **Maxflow-Mincut**
  - Maxflow-Mincut successfully passed all the test cases listed in the result table.
  - The test size increases from 6 vertices and 10 edges in mf_06.txt to 100,000 vertices and 120,000 edges in mf_100000.txt.
  - The execution time generally increases as the number of vertices and edges increases, from 0.0086 ms for mf_06.txt to 100.234 ms for mf_100000.txt.
  - The maximum-flow value is equal to the minimum-cut capacity for every test case in the result table.
  - For example, the maximum flow and minimum cut are both 23 for mf_06.txt, both 83 for mf_100.txt, and both 40 for mf_100000.txt.
  - The results demonstrate that the implementation is able to process the larger test case containing 100,000 vertices and 120,000 edges.
  - The increase in execution time for larger networks is expected because more vertices and edges must be processed during BFS, DFS, and residual-graph operations.
  - Overall, the results show that the Maxflow-Mincut implementation maintains the required correctness condition while handling increasingly large flow networks.